Visual Python Learning

Course Content Audit Report

Generated: 1/31/2026, 10:38:56 PM

Executive Summary

Total Chapters: 30
Total Sections: 97
Total Lessons: 252
Published Lessons: 252
Total Exercises: 0
Lessons with Exercises: 0
Lessons with Code Examples: 224
Total Content: 334K characters

Chapter Overview

%Ð Ch 1: Getting Started (5 lessons, 0 exercises)
%Ð Ch 2: Introduction to Python (10 lessons, 0 exercises)
%Ð Ch 3: Some Simple Numerical Programs (10 lessons, 0 exercises)
%Ð Ch 4: Functions, Scoping, and Abstraction (9 lessons, 0 exercises)
%Ð Ch 5: Structured Types, Mutability, and Higher-Order Functions (10 lessons, 0 exercises)
%Ð Ch 6: Testing and Debugging (8 lessons, 0 exercises)
%Ð Ch 7: Exceptions and Assertions (8 lessons, 0 exercises)
%Ð Ch 8: Classes and Object-Oriented Programming (12 lessons, 0 exercises)
%Ð Ch 9: A Simplistic Introduction to Algorithmic Complexity (8 lessons, 0 exercises)
%Ð Ch 10: Some Simple Algorithms and Data Structures (11 lessons, 0 exercises)
%Ð Ch 11: Plotting and More About Classes (10 lessons, 0 exercises)
%Ð Ch 12: Stochastic Programs, Probability, and Distributions (8 lessons, 0 exercises)
%Ð Ch 13: Statistical Thinking (6 lessons, 0 exercises)
%Ð Ch 14: Sampling and Confidence Intervals (5 lessons, 0 exercises)
%Ð Ch 15: Understanding Experimental Data (8 lessons, 0 exercises)
%Ð Ch 16: Random Walks and More About Data Visualization (8 lessons, 0 exercises)
%Ð Ch 17: Knapsack and Graph Optimization Problems (10 lessons, 0 exercises)
%Ð Ch 18: Dynamic Programming (8 lessons, 0 exercises)
%Ð Ch 19: Introduction to Machine Learning (10 lessons, 0 exercises)
%Ð Ch 20: Conditional Probability and Bayesian Statistics (8 lessons, 0 exercises)
%Ð Ch 21: Lies, Damned Lies, and Statistics (8 lessons, 0 exercises)
%Ð Ch 22: Classification Methods (10 lessons, 0 exercises)
%Ð Ch 23: Clustering (9 lessons, 0 exercises)
%Ð Ch 24: Working with Large Data Sets (10 lessons, 0 exercises)
%Ð Ch 25: Introduction to Machine Learning (13 lessons, 0 exercises)
%Ð Ch 26: Perceptrons & Linear Models (6 lessons, 0 exercises)
%Ð Ch 27: Neural Networks & Backpropagation (6 lessons, 0 exercises)
%Ð Ch 28: Deep Learning (6 lessons, 0 exercises)
%Ð Ch 29: Practical ML with Scikit-Learn (6 lessons, 0 exercises)
%Ð Ch 30: PyTorch Fundamentals (6 lessons, 0 exercises)

Chapter 1: Getting Started

Introduction to computation and Python basics. Learn what computation means, how to work with Python objects and expressions, and write your first programs.

Objectives:

- Understand what computation and programming mean
- Learn Python's basic data types and operators
- Master variables and assignment
- Write simple branching programs with conditionals

Lessons:

%Þ What is Computation?

Slug: what-is-computation | Content: 1058 chars | Exercises: 0

- Understand what computation means
- Distinguish declarative from imperative knowledge

%Ë Objects and Types

Slug: objects-and-types | Content: 910 chars | Exercises: 0

- Understand that everything in Python is an object
- Learn the basic data types: int, float, str, bool

%Þ Expressions and Operators

Slug: expressions-and-operators | Content: 1001 chars | Exercises: 0

- Write arithmetic expressions using operators
- Understand operator precedence

%Ë Variables and Assignment

Slug: variables-and-assignment | Content: 959 chars | Exercises: 0

- Create variables using assignment statements
- Understand how variables point to values in memory

%Ë Conditional Statements

Slug: conditional-statements | Content: 928 chars | Exercises: 0

- Write if statements to make decisions
- Use comparison operators

Chapter 2: Introduction to Python

Master strings, input/output, and create your first interactive programs. Learn text processing fundamentals used in every Python project.

Objectives:

- Master string creation and manipulation
- Handle user input and output effectively
- Understand character encoding concepts
- Use functions with arguments and defaults

Lessons:

%Đ String Basics - Creating and Printing Strings

Slug: string-basics | Content: 2442 chars | Exercises: 0

- Create strings using single, double, and triple quotes
- Understand strings as sequences of characters

%Đ String Operations - Concatenation, Repetition, and Length

Slug: string-operations | Content: 2337 chars | Exercises: 0

- Concatenate strings with the + operator
- Repeat strings with the * operator

%Đ String Indexing and Slicing

Slug: string-indexing-slicing | Content: 2275 chars | Exercises: 0

- Access individual characters using indexing
- Use positive and negative indices

%Đ String Methods Part 1 - Case and Whitespace

Slug: string-methods-part1 | Content: 2611 chars | Exercises: 0

- Use case conversion methods: upper(), lower(), title(), capitalize()
- Remove whitespace with strip(), lstrip(), rstrip()

%Đ String Methods Part 2 - Find, Replace, Split, Join

Slug: string-methods-part2 | Content: 2425 chars | Exercises: 0

- Find substrings with find(), index(), count()
- Replace text with replace()

%Đ String Immutability

Slug: string-immutability | Content: 2430 chars | Exercises: 0

- Understand that strings cannot be changed after creation
- Learn why immutability matters

%Ë Output with print()

Slug: output-with-print | Content: 968 chars | Exercises: 0

- Use print() to display output
- Print multiple values with separators

%Ë Getting Input from Users

Slug: input-function | Content: 763 chars | Exercises: 0

- Use input() to get user input
- Understand that input() always returns a string

%Ë String Formatting with f-strings

Slug: string-formatting-fstrings | Content: 696 chars | Exercises: 0

- Use f-strings for clean string formatting
- Embed expressions inside f-strings

%Đ Character Encoding

Slug: character-encoding | Content: 1403 chars | Exercises: 0

- Understand how computers represent text as numbers
- Know the basics of ASCII and Unicode

Chapter 3: Some Simple Numerical Programs

Learn fundamental algorithms for solving numerical problems including exhaustive enumeration, bisection search, and Newton-Raphson method.

Objectives:

- Understand exhaustive enumeration (brute force) approach
- Master for loops and the range() function
- Implement bisection search for finding solutions
- Understand floating-point representation and precision
- Apply Newton-Raphson method for root finding

Lessons:

%Đ Exhaustive Enumeration

Slug: exhaustive-enumeration | Content: 1593 chars | Exercises: 0
- Understand the guess-and-check approach
- Implement exhaustive search algorithms

%Đ Introduction to For Loops

Slug: introduction-to-for-loops | Content: 1579 chars | Exercises: 0
- Understand for loop syntax and structure
- Use for loops to repeat operations

%Đ The range() Function in Detail

Slug: range-function-detail | Content: 1351 chars | Exercises: 0
- Master all three forms of range()
- Use negative steps for counting backwards

%Đ Nested Loops

Slug: nested-loops | Content: 1403 chars | Exercises: 0
- Understand loops inside loops
- Trace execution of nested loops

%Đ Approximate Solutions and Epsilon

Slug: approximate-solutions-epsilon | Content: 1440 chars | Exercises: 0
- Understand why exact solutions don't always exist
- Use epsilon for acceptable error tolerance

%Đ Bisection Search Algorithm

Slug: bisection-search | Content: 1591 chars | Exercises: 0
- Understand the bisection (binary search) concept
- Implement bisection search for finding roots

%Đ Floating Point Representation

Slug: floating-point-representation | Content: 1284 chars | Exercises: 0
- Understand how computers store decimal numbers
- Recognize floating point limitations

%Đ Comparing Floating Point Numbers

Slug: comparing-floats | Content: 1229 chars | Exercises: 0
- Understand why == fails with floats
- Use epsilon-based comparison correctly

%Đ Newton-Raphson Method

Slug: newton-raphson-method | Content: 1114 chars | Exercises: 0
- Understand the Newton-Raphson algorithm
- Apply the method to find square roots

%Đ Comparing Algorithm Efficiency

Slug: algorithm-efficiency-comparison | Content: 1215 chars | Exercises: 0
- Compare exhaustive, bisection, and Newton-Raphson
- Understand Big-O notation basics

Chapter 4: Functions, Scoping, and Abstraction

Learn to write reusable code with functions, understand variable scope, and master recursion.

Objectives:

- Define and call functions with parameters and return values
- Use keyword arguments and default parameter values
- Understand local vs global scope
- Write clear function specifications with docstrings
- Implement recursive algorithms
- Work with modules and imports

Lessons:

%Đ Function Basics - Defining and Calling

Slug: function-basics | Content: 1474 chars | Exercises: 0
- Define functions using def keyword
- Call functions to execute their code

%Đ Parameters and Arguments

Slug: parameters-and-arguments | Content: 1519 chars | Exercises: 0
- Define functions with parameters
- Pass arguments when calling functions

%Đ Return Values

Slug: return-values | Content: 1455 chars | Exercises: 0
- Use return to send values back from functions
- Capture return values in variables

%Đ Keyword Arguments and Default Values

Slug: keyword-arguments-defaults | Content: 1549 chars | Exercises: 0
- Use keyword arguments for clarity
- Define parameters with default values

%Đ Local and Global Scope

Slug: local-and-global-scope | Content: 1546 chars | Exercises: 0
- Understand local variables inside functions
- Understand global variables outside functions

%Đ The global Keyword

Slug: global-keyword | Content: 1436 chars | Exercises: 0
- Modify global variables from functions
- Use the global keyword correctly

%Đ Docstrings and Specifications

Slug: docstrings-specifications | Content: 1518 chars | Exercises: 0
- Write docstrings to document functions
- Specify function inputs, outputs, and behavior

%Đ Introduction to Recursion

Slug: recursion-basics | Content: 1570 chars | Exercises: 0
- Understand recursion - functions calling themselves
- Identify base cases and recursive cases

%Đ Fibonacci and Recursion Patterns

Slug: fibonacci-recursion-patterns | Content: 1491 chars | Exercises: 0
- Implement Fibonacci recursively
- Understand multiple recursive calls

Chapter 5: Structured Types, Mutability, and Higher-Order Functions

Master Python's built-in data structures including tuples, lists, dictionaries, and sets. Understand mutability and learn functional programming concepts.

Objectives:

- Work with tuples and understand immutability
- Master lists and understand mutability and aliasing
- Use dictionaries for key-value storage
- Apply set operations for unique collections
- Write list comprehensions for elegant code
- Use higher-order functions and lambdas

Lessons:

%Đ Tuples - Immutable Sequences

Slug: tuples-basics | Content: 1196 chars | Exercises: 0

- Create and access tuples
- Understand tuple immutability

%Đ Introduction to Lists

Slug: lists-introduction | Content: 1038 chars | Exercises: 0

- Create and manipulate lists
- Understand list mutability

%Đ List Methods

Slug: list-methods | Content: 1282 chars | Exercises: 0

- Use append, extend, and insert to add items
- Use remove, pop, and clear to delete items

%Đ Aliasing and Cloning

Slug: aliasing-cloning | Content: 1426 chars | Exercises: 0

- Understand aliasing - multiple names for same list
- Recognize aliasing bugs

%Đ Dictionary Basics

Slug: dictionaries-basics | Content: 1183 chars | Exercises: 0

- Create dictionaries with key-value pairs
- Access, add, and modify values

%Đ Dictionary Methods and Iteration

Slug: dictionary-methods-iteration | Content: 1191 chars | Exercises: 0

- Use keys(), values(), and items() methods
- Iterate over dictionaries effectively

%Đ Sets - Unique Collections

Slug: sets-basics | Content: 1161 chars | Exercises: 0

- Create sets and understand uniqueness
- Add and remove elements

%Đ List Comprehensions

Slug: list-comprehensions | Content: 1136 chars | Exercises: 0

- Create lists with comprehension syntax
- Add conditions to filter elements

%Đ Functions as Objects

Slug: functions-as-objects | Content: 1177 chars | Exercises: 0

- Understand functions are first-class objects
- Assign functions to variables

%Đ Map, Filter, and Lambda

Slug: map-filter-lambda | Content: 1125 chars | Exercises: 0

- Use map() to transform sequences
- Use filter() to select elements

Chapter 6: Testing and Debugging

Learn essential skills for finding and fixing bugs, writing effective tests, and using defensive programming techniques.

Objectives:

- Understand why testing is crucial for software quality
- Design effective test cases using black-box and glass-box techniques
- Apply systematic debugging strategies
- Identify and fix common Python bugs
- Use assertions for defensive programming

Lessons:

%Đ Why Testing Matters

Slug: why-testing-matters | Content: 1454 chars | Exercises: 0
- Understand why all programs have bugs
- Recognize testing vs debugging difference

%Đ Designing Test Cases

Slug: designing-test-cases | Content: 1427 chars | Exercises: 0
- Design tests for boundary conditions
- Create normal case tests

%Đ Black-Box Testing

Slug: black-box-testing | Content: 1357 chars | Exercises: 0
- Understand black-box testing concept
- Write tests from specifications only

%Đ Glass-Box Testing

Slug: glass-box-testing | Content: 1276 chars | Exercises: 0
- Understand glass-box (white-box) testing
- Achieve code path coverage

%Đ The Debugging Mindset

Slug: debugging-mindset | Content: 1237 chars | Exercises: 0
- Approach debugging systematically
- Apply scientific method to bugs

%Đ Print Debugging

Slug: print-debugging | Content: 1233 chars | Exercises: 0
- Use print statements strategically
- Track variable values through execution

%Đ Common Python Bugs

Slug: common-python-bugs | Content: 1129 chars | Exercises: 0
- Recognize common Python errors
- Understand error messages

%Đ Assertions for Defensive Programming

Slug: assertions | Content: 1647 chars | Exercises: 0
- Use assert statement syntax
- Check preconditions and postconditions

Chapter 7: Exceptions and Assertions

Learn to handle errors gracefully with exceptions, raise your own exceptions, and use assertions for defensive programming.

Objectives:

- Handle exceptions with try/except blocks
- Understand different exception types
- Use else and finally clauses appropriately
- Raise exceptions to signal errors
- Create custom exception classes
- Use assertions effectively for debugging

Lessons:

%Đ Understanding Exceptions

Slug: understanding-exceptions | Content: 1471 chars | Exercises: 0

- Understand what exceptions are
- Know why exceptions are better than error codes

%Đ Try/Except Blocks

Slug: try-except-blocks | Content: 1246 chars | Exercises: 0

- Write try/except blocks
- Handle specific exception types

%Đ Specific Exception Types

Slug: specific-exception-types | Content: 1468 chars | Exercises: 0

- Know when each exception type occurs
- Handle ValueError for invalid values

%Đ Multiple Except Clauses

Slug: multiple-except-clauses | Content: 1348 chars | Exercises: 0

- Handle different exceptions differently
- Order exceptions from specific to general

%Đ Else and Finally Clauses

Slug: else-and-finally | Content: 1205 chars | Exercises: 0

- Use else for success-only code
- Use finally for cleanup code

%Đ Raising Exceptions

Slug: raising-exceptions | Content: 1364 chars | Exercises: 0

- Use raise to signal errors
- Choose appropriate exception types

%Đ Custom Exceptions

Slug: custom-exceptions | Content: 1432 chars | Exercises: 0

- Create custom exception classes
- Inherit from Exception properly

%Đ Assertions vs Exceptions

Slug: assertions-vs-exceptions | Content: 1426 chars | Exercises: 0

- Know when to use assertions vs exceptions
- Use assertions for programmer errors

Chapter 8: Classes and Object-Oriented Programming

Master object-oriented programming with classes, inheritance, encapsulation, and generators.

Objectives:

- Define classes with attributes and methods
- Understand __init__ constructors and self
- Use inheritance to create class hierarchies
- Override methods in subclasses
- Apply encapsulation principles
- Create generators with yield

Lessons:

%Đ Introduction to Classes and Objects

Slug: intro-classes-objects | Content: 1500 chars | Exercises: 0

- Understand what objects are (data + behavior)
- Understand what classes are (templates)

%Đ Defining Your First Class

Slug: defining-first-class | Content: 1133 chars | Exercises: 0

- Use the class keyword
- Follow naming conventions

%Đ The __init__ Constructor

Slug: init-constructor | Content: 1272 chars | Exercises: 0

- Understand what __init__ does
- Use self to reference the instance

%Đ Instance Methods

Slug: instance-methods | Content: 1294 chars | Exercises: 0

- Define methods inside classes
- Use self to access instance data

%Đ Instance Variables

Slug: instance-variables | Content: 1446 chars | Exercises: 0

- Understand instance variables belong to objects
- See each instance has independent data

%Đ Inheritance Basics

Slug: inheritance-basics | Content: 1434 chars | Exercises: 0

- Understand parent and child classes
- Inherit attributes and methods

%Đ Method Overriding

Slug: method-overriding | Content: 1271 chars | Exercises: 0

- Override parent methods in child classes
- Use super() to call parent methods

%Đ Encapsulation and Information Hiding

Slug: encapsulation | Content: 1679 chars | Exercises: 0

- Understand encapsulation concept
- Use underscore conventions for privacy

%Đ Special Methods

Slug: special-methods | Content: 1228 chars | Exercises: 0

- Implement __str__ for string representation
- Implement __repr__ for debugging

%Đ Generators and yield

Slug: generators-yield | Content: 1292 chars | Exercises: 0

- Understand generators vs regular functions
- Use yield to produce values lazily

%Đ Class Design Principles

Slug: class-design-principles | Content: 1614 chars | Exercises: 0

- Apply single responsibility principle
- Understand cohesion and coupling

%Đ Practical OOP Examples

Slug: practical-oop-examples | Content: 1146 chars | Exercises: 0

- Build complete class implementations
- Model real-world entities effectively

Chapter 9: A Simplistic Introduction to Algorithmic Complexity

Learn to analyze algorithm efficiency using Big O notation and understand the importance of choosing the right algorithm.

Objectives:

- Understand why algorithm efficiency matters
- Use Big O notation to describe complexity
- Recognize common complexity classes
- Analyze code to determine its complexity
- Choose appropriate algorithms for different situations

Lessons:

%Đ Why Complexity Matters

Slug: why-complexity-matters | Content: 1680 chars | Exercises: 0
- See dramatic performance differences between algorithms
- Understand scalability and why it matters

%Đ Measuring Algorithm Efficiency

Slug: measuring-efficiency | Content: 1411 chars | Exercises: 0
- Count operations instead of timing
- Understand why timing is unreliable

%Đ Introduction to Big O Notation

Slug: intro-big-o | Content: 1214 chars | Exercises: 0
- Understand what Big O notation represents
- Read and write Big O expressions

%Đ Common Complexity Classes

Slug: common-complexity-classes | Content: 1668 chars | Exercises: 0
- Know the most common complexity classes
- Recognize code patterns for each class

%Đ Analyzing Simple Code

Slug: analyzing-simple-code | Content: 1129 chars | Exercises: 0
- Analyze loops to determine complexity
- Handle sequential code sections

%Đ Comparing Algorithm Efficiency

Slug: comparing-algorithm-efficiency | Content: 1225 chars | Exercises: 0
- Compare algorithms solving the same problem
- Choose the best algorithm for a situation

%Đ Space Complexity

Slug: space-complexity | Content: 1309 chars | Exercises: 0
- Understand space complexity concept
- Analyze memory usage of algorithms

%Đ Best, Average, Worst Case

Slug: best-average-worst-case | Content: 1219 chars | Exercises: 0
- Understand best, average, and worst case
- Know which case Big O typically describes

Chapter 10: Some Simple Algorithms and Data Structures

Master classic algorithms and data structures - search, sort, and hash tables. These building blocks appear everywhere in software and interviews.

Objectives:

- Implement linear and binary search
- Understand sorting algorithms and their tradeoffs
- Use hash tables for O(1) lookups
- Choose the right data structure for each problem

Lessons:

%Đ Linear Search Algorithm

Slug: linear-search | Content: 1298 chars | Exercises: 0
- Understand linear search algorithm
- Implement linear search in Python

%Đ Binary Search Algorithm

Slug: binary-search | Content: 1496 chars | Exercises: 0
- Understand divide and conquer approach
- Implement binary search in Python

%Đ Search Comparison

Slug: search-comparison | Content: 1333 chars | Exercises: 0
- Compare linear and binary search performance
- Know when to use each algorithm

%Đ Introduction to Sorting

Slug: intro-sorting | Content: 1376 chars | Exercises: 0
- Understand why sorting is important
- Know the sorting problem definition

%Đ Selection Sort

Slug: selection-sort | Content: 1317 chars | Exercises: 0
- Understand selection sort algorithm
- Implement selection sort in Python

%Đ Merge Sort

Slug: merge-sort | Content: 1435 chars | Exercises: 0
- Understand divide and conquer sorting
- Implement merge sort in Python

%Đ Comparing Sorting Algorithms

Slug: comparing-sorting | Content: 1336 chars | Exercises: 0
- Compare different sorting algorithms
- Know when to use each algorithm

%Đ Hash Tables Introduction

Slug: hash-tables-intro | Content: 1158 chars | Exercises: 0
- Understand hash table concept
- Know how hash functions work

%Đ Python Dictionaries as Hash Tables

Slug: python-dictionaries | Content: 1213 chars | Exercises: 0
- Understand dict is a hash table
- Know dict time complexities

%Ë Sets and Hash-based Data Structures

Slug: sets-hash-structures | Content: 958 chars | Exercises: 0
- Understand sets as hash tables
- Use sets for O(1) membership testing

%Đ Choosing Data Structures

Slug: choosing-data-structures | Content: 1240 chars | Exercises: 0
- Compare list, dict, and set tradeoffs
- Choose best structure for each problem

Chapter 11: Plotting and More About Classes

Master data visualization with matplotlib and deepen your understanding of object-oriented programming by subclassing built-in types.

Objectives:

- Create line plots, scatter plots, and bar charts using matplotlib
- Customize plot appearance with colors, labels, and legends
- Work with multiple plots and subplots
- Apply plotting to real-world data analysis problems
- Subclass built-in types like list and dict
- Override methods to create custom data structures

Lessons:

%Đ Introduction to Matplotlib

Slug: intro-matplotlib | Content: 1207 chars | Exercises: 0
- Understand why data visualization matters
- Install and import matplotlib

%Đ Line Plots

Slug: line-plots | Content: 1284 chars | Exercises: 0
- Master plt.plot() options
- Customize colors, styles, and markers

%Đ Scatter Plots and Bar Charts

Slug: scatter-bar-plots | Content: 1148 chars | Exercises: 0
- Create scatter plots for relationships
- Create bar charts for categorical data

%Đ Multiple Datasets

Slug: multiple-datasets | Content: 1077 chars | Exercises: 0
- Plot multiple lines on one chart
- Create effective legends

%Đ Subplots

Slug: subplots | Content: 1182 chars | Exercises: 0
- Create multiple plots in one figure
- Use subplot grid layouts

%Đ Real-World Example: Mortgage Analysis

Slug: mortgages-example | Content: 1111 chars | Exercises: 0
- Apply plotting to real financial data
- Calculate mortgage payments

%Đ Data Visualization Best Practices

Slug: visualization-best-practices | Content: 1128 chars | Exercises: 0
- Choose appropriate chart types
- Create clear and honest visualizations

%Đ Subclassing Built-in Types

Slug: subclassing-builtins | Content: 1377 chars | Exercises: 0
- Extend list with custom behavior
- Extend dict with custom behavior

%Đ Advanced OOP Patterns

Slug: advanced-oop-patterns | Content: 1285 chars | Exercises: 0
- Understand composition vs inheritance
- Know basics of multiple inheritance

%Đ Putting It Together

Slug: putting-together-plotting-classes | Content: 1180 chars | Exercises: 0
- Combine classes with visualization
- Build complete data analysis workflow

Chapter 12: Stochastic Programs, Probability, and Distributions

Learn probabilistic programming, random walks, Monte Carlo simulation, and statistical inference.

Objectives:

- Understand deterministic vs stochastic programs
- Master Python's random module
- Implement random walks in 1D and 2D
- Calculate probabilities through simulation
- Apply Monte Carlo simulation methods

Lessons:

%Ð Introduction to Randomness and Stochastic Programs

Slug: intro-randomness-stochastic-programs | Content: 1249 chars | Exercises: 0

- Understand deterministic vs stochastic programs
- Learn about pseudo-random number generators

%Ë Random Walks - 1D

Slug: random-walks-1d | Content: 970 chars | Exercises: 0

- Understand the concept of a random walk
- Implement a simple 1D random walk

%Ë Random Walks - 2D

Slug: random-walks-2d | Content: 938 chars | Exercises: 0

- Extend random walks to two dimensions
- Implement 2D walks with four directions

%Ë Introduction to Probability

Slug: intro-probability | Content: 976 chars | Exercises: 0

- Understand basic probability concepts
- Calculate simple probabilities

%Ë Probability Distributions

Slug: probability-distributions | Content: 761 chars | Exercises: 0

- Understand probability distributions
- Work with uniform and binomial distributions

%Ë Monte Carlo Simulation

Slug: monte-carlo-simulation | Content: 754 chars | Exercises: 0

- Understand Monte Carlo simulation
- Estimate pi using random sampling

%Ë Statistical Inference from Simulations

Slug: statistical-inference-simulation | Content: 540 chars | Exercises: 0

- Use simulation for statistical inference
- Calculate mean and standard deviation

%Ë Advanced Monte Carlo Applications

Slug: advanced-monte-carlo | Content: 595 chars | Exercises: 0

- Apply Monte Carlo to real-world problems
- Simulate stock prices

Chapter 13: Statistical Thinking

Learn fundamental statistical concepts essential for data analysis, scientific computing, and machine learning. Master measures of central tendency, variability, distributions, and statistical significance.

Objectives:

- Calculate and interpret mean, median, and mode
- Understand variance and standard deviation
- Work with probability distributions
- Apply statistical significance testing
- Make data-driven decisions with confidence

Lessons:

%Đ Measures of Central Tendency - Mean, Median, Mode

Slug: measures-central-tendency | Content: 1344 chars | Exercises: 0

- Understand and calculate mean (average)
- Understand and calculate median (middle value)

%Đ Measures of Variability - Range, Variance, Standard Deviation

Slug: measures-variability | Content: 1305 chars | Exercises: 0

- Understand why variability matters
- Calculate range, variance, and standard deviation

%Đ Introduction to Distributions

Slug: intro-distributions | Content: 1082 chars | Exercises: 0

- Understand what a probability distribution is
- Distinguish discrete vs continuous distributions

%Đ The Normal Distribution

Slug: normal-distribution | Content: 1130 chars | Exercises: 0

- Understand the normal (Gaussian) distribution
- Apply the 68-95-99.7 rule

%Đ Statistical Significance

Slug: statistical-significance | Content: 1268 chars | Exercises: 0

- Understand what statistical significance means
- Learn about null hypotheses and p-values

%Đ Confidence Intervals and Standard Error

Slug: confidence-intervals | Content: 1050 chars | Exercises: 0

- Understand standard error of the mean
- Calculate and interpret confidence intervals

Chapter 14: Sampling and Confidence Intervals

Learn how to make inferences about populations from samples. Master the Central Limit Theorem, confidence intervals, and margin of error.

Objectives:

- Distinguish between population and sample
- Understand and apply the Central Limit Theorem
- Calculate and interpret confidence intervals
- Determine appropriate sample sizes
- Use bootstrap methods for inference

Lessons:

%Ð Population vs Sample - Fundamentals

Slug: population-vs-sample | Content: 1173 chars | Exercises: 0

- Distinguish between population and sample
- Understand representative sampling

%Ë The Central Limit Theorem

Slug: central-limit-theorem | Content: 961 chars | Exercises: 0

- Understand the Central Limit Theorem statement
- Demonstrate CLT through simulation

%Ë Confidence Intervals - Construction and Interpretation

Slug: confidence-intervals-intro | Content: 866 chars | Exercises: 0

- Understand what confidence intervals represent
- Calculate confidence intervals for means

%Ë Margin of Error and Sample Size Determination

Slug: margin-of-error-sample-size | Content: 801 chars | Exercises: 0

- Understand margin of error
- Calculate required sample size for desired precision

%Ë The Bootstrap Method

Slug: bootstrap-method | Content: 820 chars | Exercises: 0

- Understand bootstrap resampling
- Implement bootstrap confidence intervals

Chapter 15: Understanding Experimental Data

Learn to work with real experimental data using linear regression. Master fitting lines to data, evaluating model quality with R², and making predictions.

Objectives:

- Understand experimental design principles
- Implement linear regression from scratch
- Calculate and interpret R-squared
- Analyze residuals for model validation
- Make predictions using regression models

Lessons:

%Ë Experimental Design Basics

Slug: experimental-design-basics | Content: 903 chars | Exercises: 0
- Understand independent vs dependent variables
- Learn principles of good experimental design

%Ë Data Collection and Quality

Slug: data-collection-quality | Content: 778 chars | Exercises: 0
- Store experimental data in Python structures
- Calculate basic statistics on data

%Ë Introduction to Linear Regression

Slug: intro-linear-regression | Content: 908 chars | Exercises: 0
- Understand what linear regression does
- Learn the equation of a line: y = mx + b

%Ë The Least Squares Method

Slug: least-squares-method | Content: 741 chars | Exercises: 0
- Understand the least squares principle
- Learn the formulas for slope and intercept

%Ë Implementing Linear Regression

Slug: implementing-linear-regression | Content: 932 chars | Exercises: 0
- Write a reusable regression function
- Make predictions with regression model

%Ë R-Squared: Goodness of Fit

Slug: r-squared-goodness-fit | Content: 747 chars | Exercises: 0
- Understand what R² measures
- Calculate R² from scratch

%Ë Residual Analysis

Slug: residual-analysis | Content: 795 chars | Exercises: 0
- Calculate and interpret residuals
- Identify patterns in residuals

%Ð Making Predictions with Regression

Slug: predictions-with-regression | Content: 1044 chars | Exercises: 0
- Use regression models for prediction
- Understand interpolation vs extrapolation

Chapter 16: Random Walks and More About Data Visualization

Extend random walk concepts with bias and drift. Master advanced data visualization techniques including multiple plots, heatmaps, and statistical analysis of simulations.

Objectives:

- Implement biased random walks with drift
- Analyze random walk statistical properties
- Create multi-panel visualizations
- Build heatmaps and density plots
- Apply random walks to real-world modeling

Lessons:

%Ë Random Walks Revisited

Slug: random-walks-revisited | Content: 880 chars | Exercises: 0

- Review 1D random walk fundamentals
- Implement efficient walk simulations

%Ë Biased Random Walks (Drift)

Slug: biased-random-walks | Content: 844 chars | Exercises: 0

- Implement biased random walks
- Understand drift and its effects

%Ë Analyzing Random Walk Properties

Slug: analyzing-walk-properties | Content: 761 chars | Exercises: 0

- Calculate variance of final position
- Analyze distribution of endpoints

%Ë 2D Random Walk Patterns

Slug: 2d-random-walk-patterns | Content: 720 chars | Exercises: 0

- Implement 2D random walks
- Track x,y positions over time

%Ë Brownian Motion Basics

Slug: brownian-motion-basics | Content: 667 chars | Exercises: 0

- Understand Brownian motion as continuous random walk
- Implement Gaussian step random walks

%Ë Advanced Matplotlib - Multiple Subplots

Slug: matplotlib-multiple-subplots | Content: 784 chars | Exercises: 0

- Create multi-panel figures with subplots
- Compare different scenarios side-by-side

%Ë Heatmaps and Density Plots

Slug: heatmaps-density-plots | Content: 731 chars | Exercises: 0

- Create 2D histograms (heatmaps)
- Visualize endpoint distributions

%Ë 3D Visualization Basics

Slug: 3d-visualization-basics | Content: 728 chars | Exercises: 0

- Create 3D plots with matplotlib
- Visualize 3D random walks

Chapter 17: Knapsack and Graph Optimization Problems

Learn fundamental optimization techniques including the knapsack problem, greedy algorithms, and graph-based algorithms like DFS, BFS, and shortest path finding.

Objectives:

- Understand optimization problem structure
- Implement greedy and exhaustive algorithms
- Represent graphs in Python
- Traverse graphs with DFS and BFS
- Find shortest paths in graphs

Lessons:

%Ð Introduction to Optimization Problems

Slug: intro-optimization-problems | Content: 1031 chars | Exercises: 0
- Understand what optimization problems are
- Identify objective functions and constraints

%Ë The Knapsack Problem

Slug: knapsack-problem | Content: 914 chars | Exercises: 0
- Understand the 0/1 knapsack problem
- Define items with weight and value

%Ë Greedy Algorithms for Knapsack

Slug: greedy-algorithms-knapsack | Content: 840 chars | Exercises: 0
- Understand the greedy approach
- Implement greedy by value, weight, and density

%Ë Brute Force vs Greedy Tradeoffs

Slug: brute-force-vs-greedy | Content: 943 chars | Exercises: 0
- Implement brute force exhaustive search
- Compare brute force and greedy results

%Ë Introduction to Graph Theory

Slug: intro-graph-theory | Content: 954 chars | Exercises: 0
- Understand nodes (vertices) and edges
- Distinguish directed vs undirected graphs

%Ë Graph Representation (Adjacency Matrix/List)

Slug: graph-representation | Content: 871 chars | Exercises: 0
- Implement adjacency matrix representation
- Implement adjacency list representation

%Ë Graph Traversal - Depth-First Search

Slug: depth-first-search | Content: 738 chars | Exercises: 0
- Understand DFS traversal strategy
- Implement recursive DFS

%Ë Graph Traversal - Breadth-First Search

Slug: breadth-first-search | Content: 981 chars | Exercises: 0
- Understand BFS traversal strategy
- Implement BFS with queue

%Ë Shortest Path Problems

Slug: shortest-path-problems | Content: 968 chars | Exercises: 0
- Understand weighted shortest path problems
- Distinguish from unweighted BFS

%Ë Dijkstra's Algorithm (Simplified)

Slug: dijkstras-algorithm | Content: 977 chars | Exercises: 0
- Understand Dijkstra's algorithm concept
- Implement simplified version

Chapter 18: Dynamic Programming

Master dynamic programming, one of the most powerful algorithm design techniques. Learn to transform exponential-time problems into polynomial-time solutions through memoization and tabulation.

Objectives:

- Understand optimal substructure and overlapping subproblems
- Implement memoization (top-down DP)
- Implement tabulation (bottom-up DP)
- Solve classic DP problems (knapsack, LCS)
- Analyze and compare DP approaches

Lessons:

%Ð Introduction to Dynamic Programming

Slug: intro-dynamic-programming | Content: 1034 chars | Exercises: 0
- Understand what dynamic programming is
- Identify optimal substructure

%Ë Fibonacci - Naive vs Memoized

Slug: fibonacci-naive-vs-memoized | Content: 894 chars | Exercises: 0
- Implement naive recursive Fibonacci
- Analyze exponential time complexity

%Ð Memoization Technique

Slug: memoization-technique | Content: 1150 chars | Exercises: 0
- Master the memoization pattern
- Apply memoization to different problems

%Ð When to Use DP (Problem Characteristics)

Slug: when-to-use-dp | Content: 1121 chars | Exercises: 0
- Identify problems suitable for DP
- Recognize DP problem patterns

%Ë 0/1 Knapsack with DP

Slug: knapsack-dp | Content: 836 chars | Exercises: 0
- Solve 0/1 knapsack optimally with DP
- Define state and recurrence relation

%Ë Longest Common Subsequence

Slug: longest-common-subsequence | Content: 832 chars | Exercises: 0
- Understand the LCS problem
- Define state for two-sequence DP

%Ð Tabulation vs Memoization

Slug: tabulation-vs-memoization | Content: 1095 chars | Exercises: 0
- Understand top-down vs bottom-up DP
- Convert memoized solution to tabulation

%Ð DP Design Process

Slug: dp-design-process | Content: 1077 chars | Exercises: 0
- Follow systematic DP design steps
- Define state clearly

Chapter 19: Introduction to Machine Learning

Learn the fundamentals of machine learning - teaching computers to learn from data. Covers supervised learning, KNN algorithm, model evaluation, and avoiding common pitfalls like overfitting.

Objectives:

- Understand supervised vs unsupervised learning
- Work with features, labels, and training data
- Implement K-Nearest Neighbors from scratch
- Evaluate models with proper train/test splits
- Recognize and avoid overfitting

Lessons:

%Đ What is Machine Learning?

Slug: what-is-machine-learning | Content: 1304 chars | Exercises: 0

- Define machine learning
- Understand learning from data vs explicit programming

%Đ Supervised vs Unsupervised Learning

Slug: supervised-unsupervised-learning | Content: 1250 chars | Exercises: 0

- Distinguish supervised from unsupervised learning
- Understand classification vs regression

%Đ Features and Feature Engineering

Slug: features-and-feature-engineering | Content: 1317 chars | Exercises: 0

- Understand features and labels
- Extract meaningful features from data

%Đ Training and Testing Data Split

Slug: training-testing-data-split | Content: 1328 chars | Exercises: 0

- Understand why we split data
- Implement train/test split

%Ë Distance Metrics (Euclidean, Manhattan)

Slug: distance-metrics | Content: 946 chars | Exercises: 0

- Understand distance as similarity measure
- Implement Euclidean distance

%Đ K-Nearest Neighbors Algorithm

Slug: knn-algorithm | Content: 1155 chars | Exercises: 0

- Understand the KNN classification algorithm
- Implement KNN from scratch

%Đ Choosing K and KNN Limitations

Slug: choosing-k-knn-limitations | Content: 1268 chars | Exercises: 0

- Understand how K affects predictions
- Choose appropriate K values

%Ë Overfitting and Underfitting

Slug: overfitting-underfitting | Content: 871 chars | Exercises: 0

- Understand overfitting (memorizing vs learning)
- Understand underfitting (too simple)

%Ë Cross-Validation

Slug: cross-validation | Content: 347 chars | Exercises: 0

- Understand why single split is limited
- Implement K-fold cross-validation

%Ë Model Evaluation Metrics

Slug: model-evaluation-metrics | Content: 325 chars | Exercises: 0

- Understand confusion matrix
- Calculate accuracy, precision, recall

Chapter 20: Conditional Probability and Bayesian Statistics

Master Bayesian thinking - updating beliefs based on evidence. Learn conditional probability, Bayes' theorem, and build a Naive Bayes spam classifier.

Objectives:

- Understand conditional probability P(A|B)
- Apply Bayes' theorem to real problems
- Distinguish prior, likelihood, and posterior
- Implement Bayesian updating
- Build a Naive Bayes classifier

Lessons:

%Ë Conditional Probability

Slug: conditional-probability | Content: 773 chars | Exercises: 0
- Understand P(A|B) notation
- Calculate conditional probabilities

%Ë Joint and Marginal Probabilities

Slug: joint-marginal-probabilities | Content: 786 chars | Exercises: 0
- Understand joint probability P(A and B)
- Calculate marginal probabilities

%Ë Bayes' Theorem Derivation

Slug: bayes-theorem-derivation | Content: 760 chars | Exercises: 0
- Derive Bayes' theorem from conditional probability
- Understand each component of the formula

%Ë Bayes' Theorem Applications (Medical Testing)

Slug: bayes-theorem-applications | Content: 837 chars | Exercises: 0
- Apply Bayes to medical test interpretation
- Understand sensitivity and specificity

%Ë Prior and Posterior Probabilities

Slug: prior-posterior-probabilities | Content: 822 chars | Exercises: 0
- Understand prior as initial belief
- See posterior as updated belief

%Ë Bayesian Updating Process

Slug: bayesian-updating-process | Content: 719 chars | Exercises: 0
- Implement iterative Bayesian updates
- Watch beliefs evolve with data

%Ë Naive Bayes Classifier

Slug: naive-bayes-classifier | Content: 775 chars | Exercises: 0
- Understand the 'naive' independence assumption
- Implement Naive Bayes from scratch

%Ë Implementing a Spam Filter

Slug: implementing-spam-filter | Content: 757 chars | Exercises: 0
- Apply Naive Bayes to text classification
- Build a working spam filter

Chapter 21: Lies, Damned Lies, and Statistics

Develop critical thinking about data. Learn to recognize statistical manipulation, common fallacies, and misleading presentations that pervade media and business.

Objectives:

- Identify common statistical errors and fallacies
- Distinguish correlation from causation
- Recognize selection and survivorship bias
- Detect misleading visualizations
- Evaluate statistical claims critically

Lessons:

%Ë Common Statistical Errors

Slug: common-statistical-errors | Content: 919 chars | Exercises: 0

- Recognize base rate neglect
- Understand sample size issues

%Ë Correlation Does Not Imply Causation

Slug: correlation-causation | Content: 905 chars | Exercises: 0

- Understand correlation vs causation
- Identify confounding variables

%Ð Selection Bias

Slug: selection-bias | Content: 1049 chars | Exercises: 0

- Understand what selection bias is
- Identify common sources of selection bias

%Ë Survivorship Bias

Slug: survivorship-bias | Content: 964 chars | Exercises: 0

- Understand survivorship bias deeply
- Recognize it in business and media

%Ð Cherry-Picking Data (P-Hacking)

Slug: cherry-picking-p-hacking | Content: 1017 chars | Exercises: 0

- Understand p-hacking and data dredging
- See how multiple testing inflates false positives

%Ë Misleading Visualizations

Slug: misleading-visualizations | Content: 883 chars | Exercises: 0

- Recognize truncated y-axes
- Spot manipulated scales

%Ë Simpson's Paradox

Slug: simpsons-paradox | Content: 839 chars | Exercises: 0

- Understand Simpson's paradox
- See how aggregation reverses trends

%Ë Critical Evaluation of Statistical Claims

Slug: critical-evaluation-statistical-claims | Content: 889 chars | Exercises: 0

- Apply a systematic checklist to evaluate claims
- Ask the right questions about statistics

Chapter 22: Classification Methods

Master supervised learning classification algorithms. Learn logistic regression, decision trees, random forests, and how to evaluate and compare classifiers.

Objectives:

- Understand classification problem types
- Master evaluation metrics (precision, recall, F1)
- Implement logistic regression and decision trees
- Use ensemble methods like random forests
- Compare and select appropriate classifiers

Lessons:

%Đ Classification Problem Types

Slug: classification-problem-types | Content: 1039 chars | Exercises: 0
- Distinguish binary vs multi-class classification
- Understand multi-label classification

%Ë Evaluation Metrics (Accuracy, Precision, Recall)

Slug: evaluation-metrics-accuracy-precision-recall | Content: 948 chars | Exercises: 0
- Calculate accuracy and understand its limitations
- Understand precision (of predictions, how many right?)

%Ë Confusion Matrix

Slug: confusion-matrix-classification | Content: 856 chars | Exercises: 0
- Understand the confusion matrix structure
- Read and interpret confusion matrices

%Ë ROC Curves and AUC

Slug: roc-curves-auc | Content: 887 chars | Exercises: 0
- Understand ROC curve construction
- Calculate TPR and FPR at different thresholds

%Ë Logistic Regression Introduction

Slug: logistic-regression-introduction | Content: 768 chars | Exercises: 0
- Understand why linear regression fails for classification
- Learn the sigmoid function

%Ë Decision Boundaries

Slug: decision-boundaries | Content: 725 chars | Exercises: 0
- Understand what a decision boundary is
- See linear vs non-linear boundaries

%Ë Decision Trees

Slug: decision-trees-classification | Content: 887 chars | Exercises: 0
- Understand decision tree structure
- Learn how splits are chosen (Gini, entropy)

%Đ Random Forests (Ensemble Methods)

Slug: random-forests-ensemble | Content: 1074 chars | Exercises: 0
- Understand ensemble learning concept
- Learn how random forests work

%Đ K-Nearest Neighbors Revisited

Slug: knn-revisited-classification | Content: 1032 chars | Exercises: 0
- Apply KNN to classification problems
- Understand the role of K in classification

%Đ Model Comparison and Selection

Slug: model-comparison-selection | Content: 1305 chars | Exercises: 0
- Compare classifiers on same dataset
- Understand bias-variance tradeoff

Chapter 23: Clustering

Master unsupervised learning through clustering algorithms. Learn K-means, hierarchical clustering, and how to discover hidden patterns in unlabeled data.

Objectives:

- Understand unsupervised learning concepts
- Implement K-means clustering from scratch
- Apply hierarchical clustering methods
- Choose optimal number of clusters
- Validate and interpret clustering results

Lessons:

%Đ Unsupervised Learning Introduction

Slug: unsupervised-learning-introduction | Content: 1069 chars | Exercises: 0

- Understand the difference from supervised learning
- Know when to use unsupervised methods

%Đ Clustering Concepts and Applications

Slug: clustering-concepts-applications | Content: 1081 chars | Exercises: 0

- Define clustering formally
- Understand intra-cluster vs inter-cluster distance

%Ë K-Means Algorithm

Slug: kmeans-algorithm | Content: 831 chars | Exercises: 0

- Understand K-means algorithm steps
- See how Lloyd's algorithm works

%Đ K-Means Implementation

Slug: kmeans-implementation | Content: 1343 chars | Exercises: 0

- Implement complete K-means from scratch
- Handle initialization strategies

%Ë Choosing K (Elbow Method)

Slug: choosing-k-elbow-method | Content: 982 chars | Exercises: 0

- Understand the problem of choosing K
- Apply the elbow method

%Ë Hierarchical Clustering

Slug: hierarchical-clustering-intro | Content: 969 chars | Exercises: 0

- Understand hierarchical clustering concept
- Know agglomerative vs divisive approaches

%Đ Linkage Methods

Slug: linkage-methods | Content: 1122 chars | Exercises: 0

- Understand different linkage methods
- Compare single, complete, and average linkage

%Đ Dendrograms

Slug: dendrograms | Content: 1013 chars | Exercises: 0

- Understand dendrogram structure
- Read and interpret dendrograms

%Đ Cluster Validation

Slug: cluster-validation | Content: 1129 chars | Exercises: 0

- Understand internal vs external validation
- Apply multiple validation metrics

Chapter 24: Working with Large Data Sets

Master professional data science tools. Learn NumPy for fast numerical computation and Pandas for powerful data manipulation - the foundation of Python data science.

Objectives:

- Understand why NumPy is essential for performance
- Master NumPy arrays and vectorized operations
- Learn Pandas DataFrames for data manipulation
- Apply data cleaning and transformation techniques
- Build complete data analysis pipelines

Lessons:

%Đ Why NumPy? Performance Revolution

Slug: why-numpy-performance | Content: 1293 chars | Exercises: 0
- Understand why Python lists are slow
- See NumPy's massive performance advantage

%Đ NumPy Arrays and Operations

Slug: numpy-arrays-operations | Content: 1061 chars | Exercises: 0
- Create NumPy arrays in various ways
- Understand array attributes

%Ë Array Broadcasting

Slug: array-broadcasting | Content: 938 chars | Exercises: 0
- Understand broadcasting rules
- Apply operations between different-shaped arrays

%Ë NumPy for Performance

Slug: numpy-performance-tips | Content: 976 chars | Exercises: 0
- Avoid common performance pitfalls
- Use views vs copies appropriately

%Ë Introduction to Pandas

Slug: introduction-to-pandas | Content: 948 chars | Exercises: 0
- Understand what Pandas is and why it's essential
- Know the difference between Series and DataFrame

%Ë DataFrames and Series

Slug: dataframes-and-series | Content: 866 chars | Exercises: 0
- Work with Series operations
- Manipulate DataFrame columns

%Đ Data Selection and Filtering

Slug: data-selection-filtering | Content: 1039 chars | Exercises: 0
- Select data using loc and iloc
- Filter rows with boolean conditions

%Ë Data Aggregation and Grouping

Slug: data-aggregation-grouping | Content: 909 chars | Exercises: 0
- Understand the split-apply-combine pattern
- Use groupby for aggregation

%Ë Data Joining and Merging

Slug: data-joining-merging | Content: 796 chars | Exercises: 0
- Understand different join types
- Merge DataFrames on keys

%Ë Real Data Pipeline Example

Slug: real-data-pipeline | Content: 921 chars | Exercises: 0
- Build a complete data analysis pipeline
- Combine all learned techniques

Chapter 25: Introduction to Machine Learning

Master the fundamentals of machine learning. Learn how computers learn from data, understand different types of ML, build evaluation pipelines, and prepare for advanced AI topics.

Objectives:

- Understand what machine learning is and when to use it
- Distinguish between supervised, unsupervised, and reinforcement learning
- Build complete ML pipelines from data to deployment
- Evaluate models using appropriate metrics
- Prevent overfitting and underfitting
- Engineer features for better model performance

Lessons:

%Ë Supervised vs Unsupervised vs Reinforcement Learning

Slug: ml-types-supervised-unsupervised-reinforcement | Content: 720 chars | Exercises: 0
- Understand the three main types of machine learning
- Identify which type fits a given problem

%Ë Features and Labels

Slug: features-and-labels | Content: 337 chars | Exercises: 0
- Understand what features and labels are
- Identify features and labels in datasets

%Ë Training, Validation, and Test Sets

Slug: train-validation-test-sets | Content: 300 chars | Exercises: 0
- Understand why we split data
- Know train/val/test purposes

%Ë Linear Regression

Slug: linear-regression | Content: 616 chars | Exercises: 0
- Understand the linear regression model
- Implement linear regression

%Ë Gradient Descent

Slug: gradient-descent | Content: 651 chars | Exercises: 0
- Understand how models learn via optimization
- Implement gradient descent

%Ë Logistic Regression

Slug: logistic-regression | Content: 369 chars | Exercises: 0
- Understand classification with logistic regression
- Use sigmoid function

%Ë K-Nearest Neighbors

Slug: k-nearest-neighbors | Content: 427 chars | Exercises: 0
- Understand KNN algorithm
- Choose optimal K

%Ë Decision Trees

Slug: decision-trees | Content: 285 chars | Exercises: 0
- Understand decision tree structure
- Know Gini impurity

%Ë Random Forests

Slug: random-forests | Content: 443 chars | Exercises: 0
- Understand ensemble learning
- Know bagging and feature randomness

%Ë Bias-Variance Tradeoff

Slug: bias-variance-tradeoff | Content: 296 chars | Exercises: 0
- Understand bias vs variance
- Diagnose underfitting/overfitting

%Ë Regularization

Slug: regularization | Content: 332 chars | Exercises: 0
- Understand L1 and L2 regularization
- Choose regularization strength

%Ë Overfitting and Underfitting

Slug: overfitting-and-underfitting | Content: 290 chars | Exercises: 0
- Diagnose overfitting/underfitting
- Use learning curves

%Ë K-Means Clustering

Slug: k-means-clustering | Content: 401 chars | Exercises: 0
- Understand K-Means algorithm
- Choose K with elbow method

Chapter 26: Perceptrons & Linear Models

The building blocks of neural networks. Learn how perceptrons work, the perceptron learning algorithm, activation functions, and why single-layer networks have limitations.

Objectives:

- Understand the perceptron as the simplest neural network
- Learn how the perceptron learning algorithm works
- Explore different activation functions
- Understand why XOR cannot be solved by single-layer networks

Lessons:

%Ð What is a Perceptron?

Slug: what-is-a-perceptron | Content: 1159 chars | Exercises: 0
- Understand the perceptron architecture
- Learn about weights, bias, and activation

%Ë Linear Decision Boundaries

Slug: linear-decision-boundaries | Content: 489 chars | Exercises: 0
- Understand what a decision boundary is
- Visualize how weights and bias shape the boundary

%Ë Perceptron Learning Algorithm

Slug: perceptron-learning-algorithm | Content: 524 chars | Exercises: 0
- Understand the perceptron learning rule
- Implement training from scratch

%Ë Activation Functions

Slug: activation-functions | Content: 684 chars | Exercises: 0
- Understand why activation functions are needed
- Compare step, sigmoid, tanh, and ReLU

%Ë The XOR Problem

Slug: xor-problem | Content: 496 chars | Exercises: 0
- Understand why XOR is not linearly separable
- Know the historical significance

%Ë Multi-Layer Perceptrons (MLPs)

Slug: multi-layer-perceptrons | Content: 574 chars | Exercises: 0
- Understand MLP architecture
- See how hidden layers solve XOR

Chapter 27: Neural Networks & Backpropagation

Deep dive into how neural networks learn. Understand forward propagation, loss functions, backpropagation, and gradient flow through deep networks.

Objectives:

- Understand neural network architecture
- Learn forward propagation computations
- Master backpropagation and the chain rule
- Recognize gradient flow problems

Lessons:

%Ë Neural Network Architecture

Slug: neural-network-architecture | Content: 705 chars | Exercises: 0

- Understand layers, neurons, and connections
- Count parameters in a network

%Ë Forward Propagation

Slug: forward-propagation | Content: 468 chars | Exercises: 0

- Compute layer-by-layer activations
- Understand matrix operations

%Ë Loss Functions

Slug: loss-functions | Content: 683 chars | Exercises: 0

- Understand different loss functions
- Choose loss for regression vs classification

%Ë Backpropagation

Slug: backpropagation | Content: 704 chars | Exercises: 0

- Understand the chain rule
- Compute gradients layer by layer

%Ë Gradient Flow Problems

Slug: gradient-flow-problems | Content: 601 chars | Exercises: 0

- Understand vanishing gradients
- Understand exploding gradients

%Ë Training Neural Networks

Slug: training-neural-networks | Content: 880 chars | Exercises: 0

- Understand the training loop
- Monitor training progress

Chapter 28: Deep Learning

Explore deep learning architectures: CNNs for images, RNNs for sequences, and modern innovations like ResNets and Transformers.

Objectives:

- Understand what makes learning 'deep'
- Learn CNN architecture for image processing
- Understand RNNs for sequential data
- Explore modern architectures

Lessons:

%Ë Introduction to Deep Learning

Slug: introduction-to-deep-learning | Content: 815 chars | Exercises: 0

- Understand what makes networks 'deep'
- Learn the history of deep learning

%Ë Convolutional Neural Networks

Slug: convolutional-neural-networks | Content: 671 chars | Exercises: 0

- Understand CNN architecture
- Learn what each layer type does

%Ë The Convolution Operation

Slug: convolution-operation | Content: 561 chars | Exercises: 0

- Understand how convolution works
- Learn about kernels/filters

%Ë Pooling Layers

Slug: pooling-layers | Content: 656 chars | Exercises: 0

- Understand pooling operations
- Compare max vs average pooling

%Ë Recurrent Neural Networks

Slug: recurrent-neural-networks | Content: 657 chars | Exercises: 0

- Understand RNN architecture
- Learn about hidden state

%Ë Modern Architectures

Slug: modern-architectures | Content: 780 chars | Exercises: 0

- Learn about ResNets and skip connections
- Understand Transformer architecture

Chapter 29: Practical ML with Scikit-Learn

Master the complete machine learning workflow using Python's most popular ML library. Learn to build production-ready models from raw data to deployment.

Objectives:

- Understand the end-to-end ML pipeline
- Master data preprocessing techniques
- Learn feature engineering strategies
- Apply hyperparameter tuning effectively

Lessons:

%Đ The ML Pipeline: From Data to Deployment

Slug: ml-pipeline-overview | Content: 5358 chars | Exercises: 0

- Understand each stage of the ML pipeline
- Know why order matters in ML workflows

%Đ Data Preprocessing: Cleaning Your Data

Slug: data-preprocessing | Content: 9358 chars | Exercises: 0

- Handle missing values using different imputation strategies
- Encode categorical variables correctly

%Đ Feature Engineering: The Art of Creating Predictive Features

Slug: feature-engineering | Content: 9811 chars | Exercises: 0

- Understand why feature engineering is often more valuable than algorithm selection
- Master common feature engineering techniques

%Đ Model Selection: Choosing the Right Algorithm

Slug: model-selection | Content: 9412 chars | Exercises: 0

- Know which algorithms work best for different problem types
- Understand the trade-offs between accuracy, speed, and interpretability

%Đ Hyperparameter Tuning: Optimizing Your Model

Slug: hyperparameter-tuning | Content: 8976 chars | Exercises: 0

- Understand the difference between parameters and hyperparameters
- Master Grid Search, Random Search, and Bayesian optimization

%Đ Complete ML Project: From Data to Deployment

Slug: complete-ml-project | Content: 9310 chars | Exercises: 0

- Apply the complete ML workflow to a real problem
- Build a customer churn prediction model

Chapter 30: PyTorch Fundamentals

Learn to build neural networks with PyTorch, the leading deep learning framework. Master tensors, autograd, and the training loop.

Objectives:

- Understand tensors and tensor operations
- Master automatic differentiation with autograd
- Build neural networks with nn.Module
- Implement the complete training loop

Lessons:

%Đ Tensors: The Foundation of Deep Learning

Slug: pytorch-tensors | Content: 7798 chars | Exercises: 0

- Understand what tensors are and why they matter
- Create tensors from data, zeros, ones, and random values

%Đ Autograd: Automatic Differentiation

Slug: pytorch-autograd | Content: 3281 chars | Exercises: 0

- Understand how PyTorch tracks computations
- Use requires_grad and backward()

%Đ Building Neural Networks with nn.Module

Slug: pytorch-nn-module | Content: 2505 chars | Exercises: 0

- Create neural networks using nn.Module
- Understand layers, activations, and architectures

%Đ The Training Loop: Putting It All Together

Slug: pytorch-training-loop | Content: 3669 chars | Exercises: 0

- Understand the complete training process
- Implement the training loop from scratch

%Đ Datasets and DataLoaders

Slug: pytorch-data-loading | Content: 2545 chars | Exercises: 0

- Create custom datasets for any data
- Use DataLoader for efficient batching

%Đ Project: Building a CNN Image Classifier

Slug: pytorch-cnn-project | Content: 5983 chars | Exercises: 0

- Build a complete CNN from scratch
- Train on real image data

Gaps Analysis

Chapters needing exercises:

- Ch 1: Getting Started (5 lessons)
- Ch 2: Introduction to Python (10 lessons)
- Ch 3: Some Simple Numerical Programs (10 lessons)
- Ch 4: Functions, Scoping, and Abstraction (9 lessons)
- Ch 5: Structured Types, Mutability, and Higher-Order Functions (10 lessons)
- Ch 6: Testing and Debugging (8 lessons)
- Ch 7: Exceptions and Assertions (8 lessons)
- Ch 8: Classes and Object-Oriented Programming (12 lessons)
- Ch 9: A Simplistic Introduction to Algorithmic Complexity (8 lessons)
- Ch 10: Some Simple Algorithms and Data Structures (11 lessons)
- Ch 11: Plotting and More About Classes (10 lessons)
- Ch 12: Stochastic Programs, Probability, and Distributions (8 lessons)
- Ch 13: Statistical Thinking (6 lessons)
- Ch 14: Sampling and Confidence Intervals (5 lessons)
- Ch 15: Understanding Experimental Data (8 lessons)
- Ch 16: Random Walks and More About Data Visualization (8 lessons)
- Ch 17: Knapsack and Graph Optimization Problems (10 lessons)
- Ch 18: Dynamic Programming (8 lessons)
- Ch 19: Introduction to Machine Learning (10 lessons)
- Ch 20: Conditional Probability and Bayesian Statistics (8 lessons)
- Ch 21: Lies, Damned Lies, and Statistics (8 lessons)
- Ch 22: Classification Methods (10 lessons)
- Ch 23: Clustering (9 lessons)
- Ch 24: Working with Large Data Sets (10 lessons)
- Ch 25: Introduction to Machine Learning (13 lessons)
- Ch 26: Perceptrons & Linear Models (6 lessons)
- Ch 27: Neural Networks & Backpropagation (6 lessons)
- Ch 28: Deep Learning (6 lessons)
- Ch 29: Practical ML with Scikit-Learn (6 lessons)
- Ch 30: PyTorch Fundamentals (6 lessons)

Legend: %ï Complete | %Ð Content only | %Ë Incomplete